Code & Data Availability Statement

Track 2: AI for MEMS

Tushar Dudeja (Student Member, IEEE) and Nawaz Shafi (Member, IEEE), Vellore Institute of Technolo

Code & Data Availability Statement

Track 2: AI for MEMS

Repository

https://github.com/thetushardudeja1/mems-differentiable-pullin — public, MIT licensed.

Run it without installing anything — click the badge, then `Runtime $\rightarrow$ Run all`:

Open in Colab: https://colab.research.google.com/github/thetushardudeja1/mems-differentiable-pullin/blob/main/notebooks/MEMS_differentiable_pullin.ipynb

The badge opens this repository's notebook directly in Google Colab. Nothing needs to be cloned or installed; the notebook clones the repository in its first cell. A free Google account and the free CPU runtime are sufficient.

The notebook executes in about four minutes on a free Colab CPU and reproduces the validation suite, the exact-gradient identity check, the optimal-exponent reversal and the inverse-design descent, live. No licence, no GPU, no dataset.

Code

All code is released in this repository under an open licence. It is self-contained: a differentiable MEMS pull-in solver, its validation suite, the inverse-design and amortized-design pipelines, the reinforcement-learning environment and policy, the surrogate baselines we benchmarked ourselves against, and the full figure/table generation chain.

Component	Files
Interactive notebook — Colab-ready, ~4 min CPU, outputs included	`notebooks/MEMS_differentiable_pullin.ipynb`
Method deep-dive notebook (~3 min CPU)	`notebooks/method_deep_dive.ipynb`
Differentiable solver (fold system, exact gradients)	`sim/beam.py`
Lumped dynamic model (GPU-batched)	`sim/pullin.py`
Validation suite	`sim/test_fold.py`, `validate_beam.py`, `validate_nazemi.py`, `validate_mtest.py`, `model2dof.py`
Inverse design	`sim/inverse_design.py`
Amortized design network	`sim/amortized_hard.py`, `amortized_design.py`, `amortized_vs_converged.py`
RL environment and policy	`sim/env2dof.py`, `train_rl2dof.py`, `analyze_rl2dof.py`, `test_env2dof.py`
Surrogate baselines (MLP, FNO)	`sim/surrogate_fair.py`, `fno_surrogate.py`, `surrogate_vs_direct.py`
Benchmarks	`sim/bench.py`, `bench_one.py`
Sensitivity map (autodiff vs. finite differences)	`sim/sensitivity_map.py`

Component	Files
Figures and tables	`sim/gen_figdata.py`, `gen_figdata2.py`, `make_figures.py`, `make_tables.py`, `make_arch.py`

Data

This project uses no training dataset. That is the central methodological point: because the physics is differentiable, every gradient comes from a live solve rather than an archive of simulations. The amortized network was trained with **6,000 live solves and 0 stored samples**; the surrogate baselines we built for comparison are the only components that consume sampled data, and those samples are generated on demand by the included scripts.

Validation targets are numerical values transcribed from published papers (geometry, material properties, measured voltages). Each value is cited at the point of use in the source file, and `papers/README.md` records what every reference is used for. **Third-party PDFs are not redistributed** — they are excluded via `.gitignore`; the index gives full citations and DOIs so any reader can obtain them.

The result arrays behind every figure (`sim/figdata_*.npz`, `sim/*.npy`, 84 kB in total) **are committed**, so the interactive notebook can display the three long experiments after a fresh clone without re-running them. They are fully regenerable with `gen_figdata.py`.

The two trained networks are committed as well — `sim/amort_hard_theta.pkl` (42 kB, the amortized design network) and `sim/rl_policy.pkl` (38 kB, the adaptive control policy). Both are small enough to publish, so the released results can be loaded and re-evaluated directly rather than only described. The training scripts regenerate them from scratch.

Reproducing everything

```
conda env create -f environment.yml
conda activate mems
cd sim

# interactive notebook: every fast experiment live, ~4 min CPU
jupyter lab ../notebooks/MEMS_differentiable_pullin.ipynb

# validation (each prints PASS/FAIL against published values)
python test_fold.py
python validate_beam.py
python model2dof.py
python validate_nazemi.py
python validate_mtest.py

# regenerate all figure data, then figures and LaTeX tables
python gen_figdata.py && python gen_figdata2.py
python make_figures.py && python make_tables.py && python make_arch.py
```

Runtime is a few minutes per validation script and roughly 30 minutes for the full figure-data regeneration, on a laptop CPU. **No commercial licence (COMSOL, ANSYS, MATLAB) is required at any point** — this is a deliberate design goal, since licence cost is a barrier to reproducing MEMS design work.

Environment

Python 3.11, JAX 0.10.2, NumPy, Matplotlib; pinned in `environment.yml`. Developed on WSL2 (Ubuntu) with an RTX 4060.

Hardware note. The static beam solver requires float64 — the fourth-order stencil is cancellation-limited and fp32 does not converge (residual 2.0 versus 1e-8) — and consumer GPUs run fp64 at roughly 1/64 of fp32 rate.

It is therefore CPU-bound by design, and results are **not** GPU-dependent. The GPU is used only for the fp32 dynamics and RL training, where it gives ~8×.

Determinism

All stochastic components use explicit, seeded JAX PRNG keys, so runs are reproducible. RL results are reported across 3 independent seeds with standard deviations, on a fixed held-out set of 512 devices.

Licence